# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## ASSESSMENT TOOL 2 – Technical Presentation Questions

| | | | |
|---|---|---|---|
| **Course Code:** | CSA0303 | **Course Title:** | Data Structures |
| **CO Assessed:** | CO1 | **Assessment:** | Assessment Tool 2 – Technical Presentations Questions |
| **Weightage:** | 10% of CIA | **Total Marks:** | 100 |
| **CO1 Statement:** | Basics, Data, Data object, Data structure, Abstract Data Types (ADT), realization of ADT in 'C', Primitive and non-primitive data structure, Linear and non-linear data structure Arrays & Recursion, Performance analysis and Measurement: Time and space analysis of algorithms, Average, best and worst-case analysis. | | |
| **Student Name:** | M. RUMANA KHAN | **Reg. No.:** | 192471025 |
| **Section / Batch:** | B.TECH CSE(BIO) | **Date:** | 15/7/26 |

**Code of Conduct**

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

**Instructions**

- This test contains 3 Concept mapping questions. Total: 100 Marks.
- When a student answer using a concept map, in evaluation the answer should contain following four requirements: Understanding of concepts, Connections between ideas, Logical structure, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- Create a PPT with 15 slides – Laptop permitted for presentation in the class. Take Print out and submit in the class.

| Section / Topic | Focus | Q Nos | Marks |
|---|---|---|---|
| Realization of ADT in 'C'- Array DS- Performance analysis and Measurement: Time and space analysis of algorithms. | Linked List data Structure | 1 | 30 |
| Primitive and non-primitive data structure, Linear and non-linear data structure Arrays | Tries data Structure | 2 | 40 |
| Dynamic linear Data Structures | Queue Data structures | 3 | 40 |
| | **GRAND TOTAL:** | **100 Marks** | |

## Annexure A – Technical Presentation

1) A music streaming application maintains a playlist where songs can be added, removed, or reordered dynamically. Recommend a suitable data structure and justify based on flexibility and efficiency. Prepare a technical Presentation

2) A dictionary application needs to quickly search whether a word exists in its database. Suggest an appropriate data structure and justify based on search performance. Prepare a technical Presentation.

3) A task scheduling system assigns jobs based on execution priority levels. Suggest a suitable data structure and justify performance efficiency.

### Rubrics for Calculation

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Problem Context | 20 | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |
| Application of Concepts | 30 | Accurately applies appropriate concepts to solve complex problems | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy | Fails to apply appropriate concepts |
| Problem-Solving Approach | 20 | Logical, well-structured, and efficient approach | Mostly logical approach with minor gaps | Basic approach with some inconsistencies | Illogical or unclear approach |
| Accuracy of Solution | 20 | Completely correct solution with proper justification | Mostly correct with minor errors | Partially correct solution | Incorrect or incomplete solution |
| Clarity | 10 | Clear, well-organized, and properly explained solution | Understandable with minor clarity issues | Limited clarity and explanation | Poorly presented and difficult to understand |

| Faculty In-charge | Course Coordinator | Program Director |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

# SIMATS ENGINEERING

# Data Structure Selection for Real-World Applications

**CSA 0303 – Data Structures**

**CO1 – AT2- TECHNICAL PRESENTATION**

**Presented by:**
M Rumana Khan (192471025)

**Guided by:**
Dr. T. Rajesh Kumar

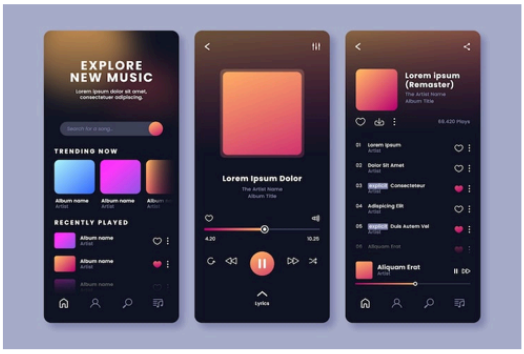
# TOPIC 1: Music Streaming Playlist

## Problem Statement

**Scenario :**

A music streaming application maintains a playlist. Users can:

- Add songs
- Remove songs
- Reorder songs
- Move to next/previous song




**Challenge**

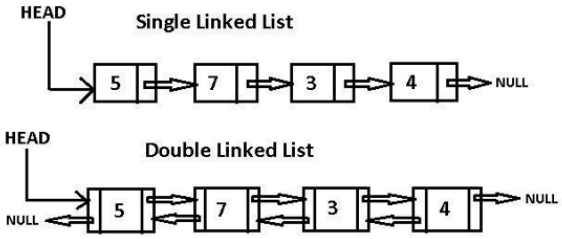
- ❑ Playlist changes happen frequently.
- ❑ Operations should be fast without shifting large amounts of data.

# Recommended Data Structure

**Doubly Linked List :**

A **Doubly Linked List** stores each song as a node containing:

- Song information
- Pointer to previous song
- Pointer to next song



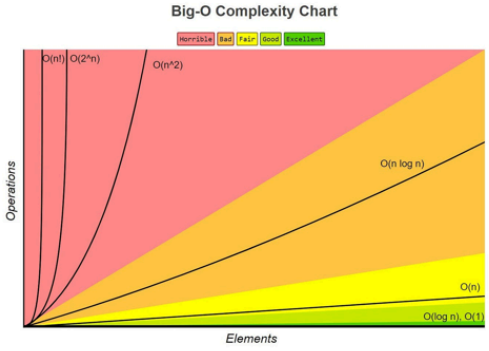

**Why Doubly Linked List?**

✓ Efficient insertion
✓ Efficient deletion
✓ Easy forward and backward navigation
✓ Supports playlist reordering

# Working & Time Complexity

**Working :**

1. Every song is linked to both its previous and next song.
2. Adding or deleting a song only requires updating the pointers.
3. No shifting of other songs is needed.



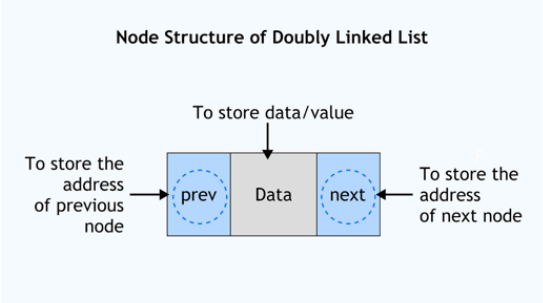

**Performance :**

- Insert Song → **O(1)**
- Delete Song → **O(1)**
- Move to Next/Previous Song → **O(1)**
- Search Song → **O(n)**

# Advantages

Compared to arrays, a **Doubly Linked List**:

- Does not require shifting elements during insertion or deletion.
- Can grow or shrink dynamically.
- Supports quick playlist editing.
- Allows smooth next and previous navigation.

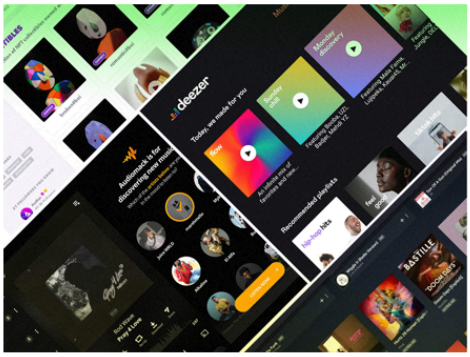



# Conclusion

**Recommended Data Structure:** Doubly Linked List.

**Justification :**

- ✓ Best suited for dynamic playlists.
- ✓ Provides efficient insertion, deletion, and reordering.
- ✓ Ensures smooth navigation between songs.



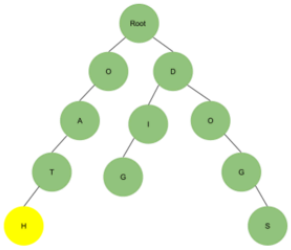

**Real-world Example :**
Spotify, Apple Music, and VLC Media Player use linked structures for playlist navigation.

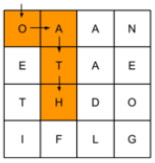
# TOPIC 2: Dictionary Word Search

## Problem Statement

**Scenario :**
A dictionary application stores millions of words and needs to determine whether a word exists in the database.

**Requirements**
- Extremely fast word lookup
- Efficient insertion of new words
- Easy deletion of obsolete words
- Ability to handle a very large dataset



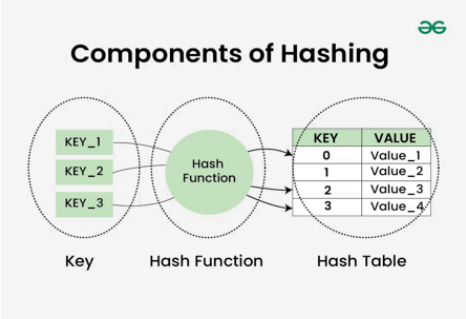




**Challenge**
Searching each word one by one (Linear Search) is inefficient because it requires **O(n)** time.

# Recommended Data Structure

**Hash Table :**
A **Hash Table** is a data structure that stores data as **key-value pairs** and uses a **hash function** to calculate the storage location (index) of each word.

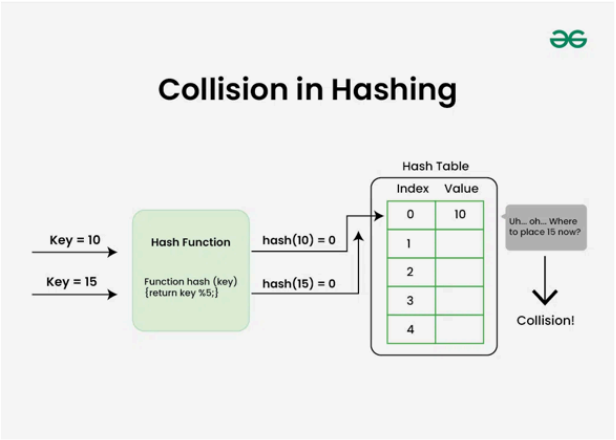
**Working :**
1. The input word is passed through a hash function.
2. The hash function generates an index.
3. The word is stored at that index in the hash table.
4. During searching, the same hash function is applied to directly access the required location.



# Working & Time Complexity

**Time Complexity :**

- Search: **O(1)** (Average)
- Insert: **O(1)** (Average)
- Delete: **O(1)** (Average)
- Worst Case: **O(n)** (due to collisions)



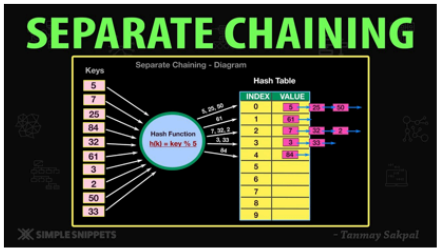

**Advantages :**

❑ Fast lookup
❑ Efficient updates
❑ Suitable for large databases

# Why Hash Table?

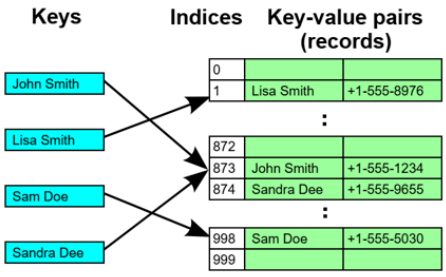
**Collision Handling**

❑ **Separate Chaining:** Store collided words in a linked list.
❑ **Open Addressing:** Find another available index.



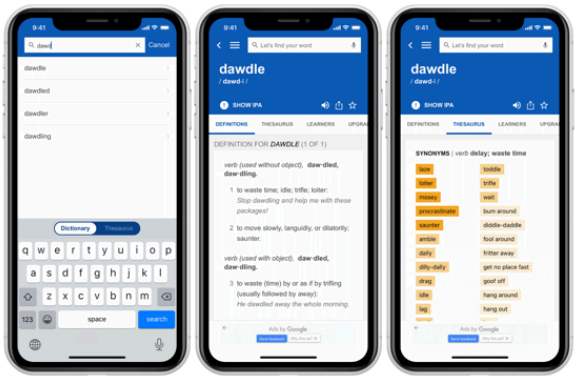

**Why Choose It?**

- Faster than arrays and linked lists.
- Better than BSTs for exact-word search.
- Simpler and more memory-efficient than Tries when prefix search isn't needed.



# Conclusion

**Recommended Data Structure:** Hash Table.




**Justification :**

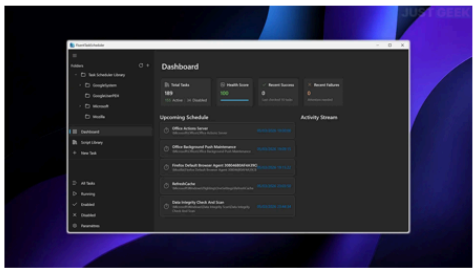
✓ Provides O(1) average search time.
✓ Handles large datasets efficiently.
✓ Best suited for exact word lookup.

**Real-world Example :**
Dictionary Apps, Spell Checkers, Password Verification, Database Indexing.

# TOPIC 3: Task Scheduling System

## Problem Statement

**Scenario :**
A task scheduler executes jobs based on their priority rather than the order in which they arrive.



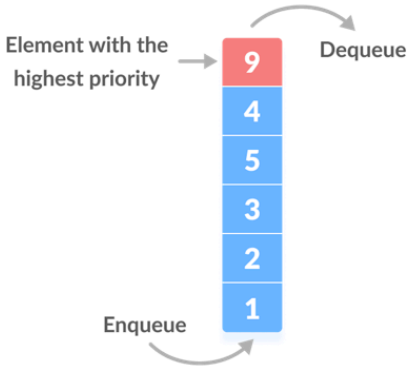

**Requirements**

- Execute the highest-priority task first.
- Insert new tasks efficiently.
- Remove completed tasks quickly.

**Challenge**
Maintain the correct priority order while handling dynamic task arrivals.

# Recommended Data Structure

**Priority Queue (Binary Heap) :**
A **Priority Queue** is a data structure where each task is assigned a priority, and the task with the highest priority is processed first.

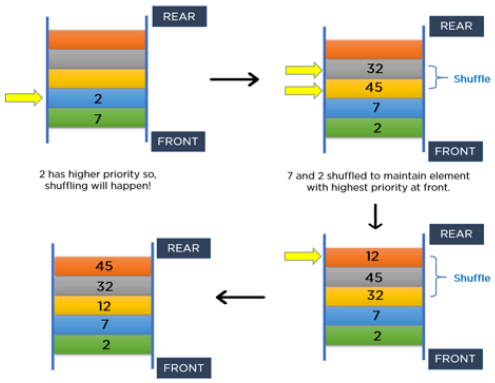



**Working :**
1. New tasks are inserted into the Binary Heap.
2. The heap automatically reorganizes to maintain the priority order.
3. The highest-priority task is always stored at the root and executed first.

**Benefit:** Provides efficient scheduling without sorting the entire task list after every insertion.

# Working & Time Complexity

**Time Complexity :**
- Insert Task → **O(log n)**
- Remove Highest Priority → **O(log n)**
- Peek Highest Priority → **O(1)**

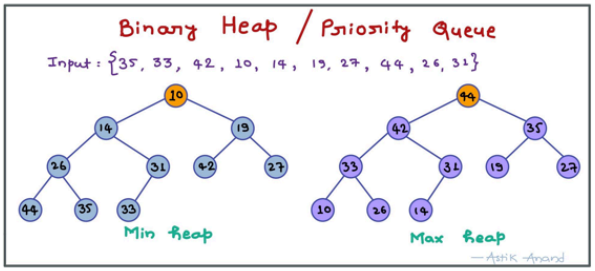



**Advantages :**
- ❑ Fast retrieval of the highest-priority task.
- ❑ Efficient insertion and deletion.
- ❑ Maintains task priorities automatically.
- ❑ Performs well even when tasks are frequently added or removed.

# Why Priority Queue ?

**Why Not a Normal Queue?**

- A normal queue follows the **First-In, First-Out (FIFO)** principle.
- A Priority Queue executes tasks based on their importance rather than arrival time.




**Why Binary Heap?**

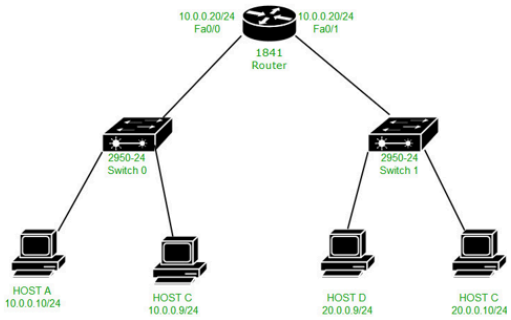
- ➢ Maintains priority efficiently.
- ➢ Requires less memory than many alternative implementations.
- ➢ Delivers balanced performance for insertion and deletion operations.

# Conclusion

**Recommendation: Priority Queue (Binary Heap)**

**Justification**

- Always executes the highest-priority task first.
- Provides efficient task management with **O(log n)** insertion and deletion.
- Suitable for systems where priorities change dynamically.




**Real-World Applications**

- ✓ CPU Process Scheduling
- ✓ Printer Job Scheduling
- ✓ Hospital Emergency Systems
- ✓ Network Packet Routing
- ✓ Event-Driven Simulations


THANK YOU!